

Лабораторная работа №7. ООП.

Выполнил лабораторную работу:

Гордеев Максим Дмитриевич

Группа: 6204-010302D

Цель лабораторной работы: Внести изменения в существующий набор типов табулированных функций, позволяющие обрабатывать точки функций по порядку (паттерн «Итератор»), а также выбирать тип объекта табулированной функции при его неявном создании (паттерн «Фабричный метод» и средства рефлексии).

Ход выполнения лабораторной работы

Задание 1:

Сделаем так, чтобы точки табулированных функций можно было перебирать в цикле **for – each**. Для этого сначала обозначим, что интерфейс табулированной функции наследуется от **Iterable<FunctionPoint>**:

```
5 public interface TabulatedFunction extends Function, Cloneable, Iterable<FunctionPoint>{
```

Затем переопределим метод **iterator()**, возвращающий объект анонимного класса, тип объекта - **Iterator<FunctionPoint>**.

Для **ArrayTabulatedFunction**:

```
333 @Override
334 public Iterator<FunctionPoint> iterator() {
335     return new Iterator<FunctionPoint>(){
336         private int index = 0;
337
338         @Override
339         public boolean hasNext(){
340             return index < size;
341         }
342
343         @Override
344         public FunctionPoint next(){
345             if (!hasNext()){
346                 throw new NoSuchElementException();
347             }
348             return points[index++];
349         }
350
351         public void delete(int index) throws UnsupportedOperationException {
352             throw new UnsupportedOperationException();
353         }
354     };
355 }
```

Для **LinkedListTabulatedFunction**:

```

625     @Override
626     public Iterator<FunctionPoint> iterator(){
627         return new Iterator<FunctionPoint>(){
628
629             private FunctionNode current = head;
630             private FunctionNode lastReturned = null;
631
632             @Override
633             public boolean hasNext(){
634                 return current != null;
635             }
636
637
638             @Override
639             public FunctionPoint next(){
640                 if (!hasNext()){
641                     throw new NoSuchElementException();
642                 }
643
644                 lastReturned = current;
645                 FunctionPoint point = current.getPoint();
646                 current = current.getNext();
647                 return point;
648             }
649
650             public void delete(int index) throws UnsupportedOperationException{
651                 throw new UnsupportedOperationException();
652             }
653         };
654     }
655 }

```

Проверим работу Итератора:

```

public class Main {
    Run | Debug
    public static void main(String[] args){
        TabulatedFunction f = new LinkedListTabulatedFunction(leftX: 1, rightX: 3, pointsCount: 5);
        for (FunctionPoint p : f){
            System.out.println(p);
        }
    }
}

```

```

    public static void main(String[] args){
        TabulatedFunction f = new ArrayTabulatedFunction(leftX: 1, rightX: 3, pointsCount: 5);
        for (FunctionPoint p : f){
            System.out.println(p);
        }
    }
}

```

Результат:

```

PS C:\Users\USER\OneDrive\Documents\Java_projects\Lab-7-2025> java Main
(1,000, 0,000)
(1,500, 0,000)
(2,000, 0,000)
(2,500, 0,000)
(3,000, 0,000)
PS C:\Users\USER\OneDrive\Documents\Java_projects\Lab-7-2025> 

```

Задание 2:

Наш утилитный класс имеет метод **tabulate()**, который не может динамически выбирать тип возвращаемой табулированной функции, поэтому необходимо применить «Фабрику» объектов.

Для этого создадим интерфейс «Фабрики» и сделаем так, чтобы наши классы **ArrayTabulatedFunction** и **LinkedListTabulatedFunction** включали в себя соответствующие вложенные классы **ArrayTabulatedFunctionFactory** и **LinkedListTabulatedFunctionFactory**, реализующие этот интерфейс.

TabulatedFunctionFactory:

```
2 package functions;
3
4 public interface TabulatedFunctionFactory {
5     TabulatedFunction createTabulatedFunction(double var1, double var3, int var5);
6
7     TabulatedFunction createTabulatedFunction(double var1, double var3, double[] var5);
8
9     TabulatedFunction createTabulatedFunction(FunctionPoint[] var1);
10 }
11
```

ArrayTabulatedFunctionFactory:

```
357 public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory {
358
359     public ArrayTabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount){
360         return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
361     }
362
363     public ArrayTabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values){
364         return new ArrayTabulatedFunction(leftX, rightX, values);
365     }
366
367     public ArrayTabulatedFunction createTabulatedFunction(FunctionPoint[] newPoints){
368         return new ArrayTabulatedFunction(newPoints);
369     }
370
371 }
372
```

LinkedListTabulatedFunctionFactory:

```
656 public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory {
657
658     public LinkedListTabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
659         return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
660     }
661
662     public LinkedListTabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
663         return new LinkedListTabulatedFunction(leftX, rightX, values);
664     }
665
666     public LinkedListTabulatedFunction createTabulatedFunction(FunctionPoint[] newPoints) {
667         return new LinkedListTabulatedFunction(newPoints);
668     }
669
670 }
671
672
```

Теперь в классе **TabulatedFunctions** создадим приватное статическое поле **factory**, содержащее объект «Фабрики». Используем функционал «Фабрики» в методе **tabulate()**:

```
private static TabulatedFunctionFactory factory = new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory();
```

```
public static void setTabulatedFunctionFactory(TabulatedFunctionFactory newFactory){  
    factory = newFactory;  
}
```

```
27 public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount){  
28     if (largeThan(rightX, function.getRightDomainBorder()) || largeThan(function.getLeftDomainBorder(), leftX)){  
29         throw new IllegalArgumentException(s: "Границы выходят за область определения функции");  
30     }  
31  
32     // Выясняем значение x для каждой точки  
33     double[] values = new double[pointsCount];  
34     double delta = (rightX - leftX) / (pointsCount - 1);  
35     for (int i = 0; i < pointsCount - 1; ++i){  
36         values[i] = function.getFunctionValue(leftX + i*delta);  
37     }  
38     values[pointsCount-1] = function.getFunctionValue(rightX);  
39  
40     //return new LinkedListTabulatedFunction(leftX, rightX, values);  
41     return factory.createTabulatedFunction(leftX, rightX, values);  
42 }  
43
```

Проверим:

```
17 Function f = new Cos();  
18 TabulatedFunction tf;  
19 tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);  
20 System.out.println(tf.getClass());  
21 TabulatedFunctions.setTabulatedFunctionFactory(new  
22 LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());  
23 tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);  
24 System.out.println(tf.getClass());  
25 TabulatedFunctions.setTabulatedFunctionFactory(new  
26 ArrayTabulatedFunction.ArrayTabulatedFunctionFactory());  
27 tf = TabulatedFunctions.tabulate(f, leftX: 0, Math.PI, pointsCount: 11);  
28 System.out.println(tf.getClass());
```

```
class functions.LinkedListTabulatedFunction  
class functions.LinkedListTabulatedFunction  
class functions.ArrayTabulatedFunction  
PS C:\Users\USER\OneDrive\Documents\Java_projects\Lab-7-2025>
```

Задание 3:

В классе **TabulatedFunctions** добавим ещё три перегруженных версии метода **createTabulatedFunction()**, у этих методов будут почти те же самые аргументы, что и в конструкторах объектов табулированных функции, но с добавлением ещё одного аргумента - ссылка **.Class**. Аналогичным образом сделаем перегрузку функции **tabulate()**:

```

128 public static TabulatedFunction createTabulatedFunction(
129     Class<? extends TabulatedFunction> functionClass,
130     double leftX, double rightX, int pointsCount) {
131
132     try {
133         Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(...parameterTypes: double.class,
134             double.class, int.class);
135
136         return constructor.newInstance(leftX, rightX, pointsCount);
137     } catch (ReflectiveOperationException e) {
138         throw new IllegalArgumentException("Ошибка в создании объекта " + functionClass.getName(), e);
139     }
140 }
141
142
143 public static TabulatedFunction createTabulatedFunction(
144     Class<? extends TabulatedFunction> functionClass,
145     double leftX, double rightX, double[] values) {
146
147     try {
148         Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(...parameterTypes: double.class,
149             double.class, double[].class);
150
151         return constructor.newInstance(leftX, rightX, values);
152     } catch (ReflectiveOperationException e) {
153         throw new IllegalArgumentException("Ошибка в создании объекта " + functionClass.getName(), e);
154     }
155 }
156
157

```

```

158 public static TabulatedFunction createTabulatedFunction(
159     Class<? extends TabulatedFunction> functionClass,
160     FunctionPoint[] points) {
161
162     try {
163         Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(...parameterTypes: FunctionPoint[].class);
164
165         return constructor.newInstance((Object) points);
166     } catch (ReflectiveOperationException e) {
167         throw new IllegalArgumentException("Ошибка в создании объекта " + functionClass.getName(), e);
168     }
169 }
170
171

```

```

173 public static TabulatedFunction tabulate(Class<? extends TabulatedFunction> functionClass,
174     Function function, double leftX, double rightX, int pointsCount) {
175
176     if (largerThan(rightX, function.getRightDomainBorder())
177         || largerThan(function.getLeftDomainBorder(), leftX)) {
178         throw new IllegalArgumentException(s: "Границы выходят за область определения функции");
179     }
180
181     try {
182         Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(...parameterTypes: double.class,
183             double.class, int.class);
184
185         TabulatedFunction tabulatedFunction = constructor.newInstance(leftX, rightX, pointsCount);
186         double delta = (rightX - leftX) / (pointsCount - 1);
187         for (int i = 0; i < pointsCount; ++i) {
188             double x = leftX + i * delta;
189             tabulatedFunction.setPointV(i, function.getFunctionValue(x));
190         }
191
192         return tabulatedFunction;
193     } catch (ReflectiveOperationException e) {
194         throw new IllegalArgumentException("Ошибка при создании объекта " + functionClass.getName(), e);
195     }
196 }
197
198

```

Проверим работу кода:

```

30     TabulatedFunction f;
31
32     f = TabulatedFunctions.createTabulatedFunction(
33         functionClass: ArrayTabulatedFunction.class, leftX: 0, rightX: 10, pointsCount: 3);
34     System.out.println(f.getClass());
35     System.out.println(f);
36
37     f = TabulatedFunctions.createTabulatedFunction(
38         functionClass: ArrayTabulatedFunction.class, leftX: 0, rightX: 10, new double[] {0, 10});
39     System.out.println(f.getClass());
40     System.out.println(f);
41
42     f = TabulatedFunctions.createTabulatedFunction(
43         functionClass: LinkedListTabulatedFunction.class,
44         new FunctionPoint[] {
45             new FunctionPoint(x: 0, y: 0),
46             new FunctionPoint(x: 10, y: 10)
47         }
48     );
49     System.out.println(f.getClass());
50     System.out.println(f);
51
52     f = TabulatedFunctions.tabulate(functionClass: LinkedListTabulatedFunction.class, new Sin(), leftX: 0, Math.PI, pointsCount: 11);
53     System.out.println(f.getClass());
54     System.out.println(f);
55     }
56
57

```

```

83     public static TabulatedFunction inputTabulatedFunction(
84         InputStream in,
85         Class<? extends TabulatedFunction> functionClass) throws IOException {
86
87         try (DataInputStream dataIn = new DataInputStream(in)) {
88             int pointsCount = dataIn.readInt();
89             FunctionPoint[] points = new FunctionPoint[pointsCount];
90
91             for (int i = 0; i < pointsCount; i++) {
92                 points[i] = new FunctionPoint(dataIn.readDouble(), dataIn.readDouble());
93             }
94
95             try {
96                 Constructor<? extends TabulatedFunction> constructor = functionClass
97                     .getConstructor(...parameterTypes: FunctionPoint[].class);
98                 return constructor.newInstance((Object) points);
99             } catch (ReflectiveOperationException e) {
100                 throw new RuntimeException(message: "Ошибка в создании табулированной функции", e);
101             }
102         }
103     }

```

```

137 public static TabulatedFunction readTabulatedFunction(
138     Reader in,
139     Class<? extends TabulatedFunction> functionClass) throws IOException {
140
141     StreamTokenizer tokenizer = new StreamTokenizer(in);
142     tokenizer.parseNumbers();
143
144     tokenizer.nextToken();
145     int pointsCount = (int) tokenizer.nval;
146     FunctionPoint[] points = new FunctionPoint[pointsCount];
147
148     for (int i = 0; i < pointsCount; ++i) {
149         tokenizer.nextToken();
150         double x = tokenizer.nval;
151
152         tokenizer.nextToken();
153         double y = tokenizer.nval;
154
155         points[i] = new FunctionPoint(x, y);
156     }
157
158     try {
159         Constructor<? extends TabulatedFunction> constructor = functionClass
160             .getConstructor(new Class[] { FunctionPoint[].class });
161         return constructor.newInstance((Object) points);
162     } catch (ReflectiveOperationException e) {
163         throw new RuntimeException(message: "Ошибка в создании табулированной функции", e);
164     }
165 }

```

Результат:

```

class functions.ArrayTabulatedFunction
{(0,000, 0,000), (5,000, 0,000), (10,000, 0,000)}
class functions.ArrayTabulatedFunction
{(0,000, 0,000), (10,000, 10,000)}
class functions.LinkedListTabulatedFunction
{(0,000, 0,000), (10,000, 10,000)}
class functions.LinkedListTabulatedFunction
{(0,000, 0,000), (0,314, 0,309), (0,628, 0,588), (0,942, 0,809), (1,257, 0,951), (1,571, 1,000), (1,885, 0,951), (2,199, 0,809), (2,513, 0,588), (2,827, 0,309),
(3,142, 0,000)}
PS C:\Users\USER\OneDrive\Documents\Java_projects\Lab-7-2025>

```

Активация Windows
Чтобы активировать Windows, перейдите в раздел
"Параметры".